



**Solar Panel Output Prediction Using ML
With Weather Data
Project Code: 25-2-D-3
Phase B - Implementation and Results**

Yovel Jirad & Shay Pinsky

Braude College Of Engineering

Capstone Project Phase B - 61999

Github Repository Link

Live Application Link

Supervisors: Dr. Dan Lemberg and Mrs. Elena Kramer

January 2026

Contents

1	Problem and Motivation	1
2	Research and Development Process	1
2.1	Research Methodology	1
2.1.1	Literature Review and Foundation	1
2.1.2	Experimental Development Environment	1
2.2	Iterative Model Optimization	2
2.2.1	Trial and Error Process	2
2.3	GRU Model Development and Improvements	2
2.3.1	Key Architectural Changes	2
2.3.2	Data Preparation Pipeline	4
2.4	Autoformer Model Development and Improvements	4
2.4.1	Architecture Selection Rationale	4
2.4.2	Key Configuration Decisions	4
2.4.3	Feature Integration	5
2.5	Summary of Model Improvements	5
3	Development Tools and Methodologies	5
3.1	Development Tools and Technologies	5
3.1.1	Machine Learning and Data Processing	6
3.1.2	Backend Development	6
3.1.3	Frontend Development	7
3.1.4	External Data Sources	7
3.1.5	Development and Collaboration Tools	7
4	architecture Diagram	7
5	Challenges, Solutions, and Results	8
5.1	Technical Challenges and Solutions	8
5.1.1	Challenge 1: Low Initial Model Performance	8
5.1.2	Challenge 2: MAPE Metric Inadequacy	9
5.2	Data and Infrastructure Challenges	10
5.2.1	Challenge 3: Absence of Real Solar Farm Data	10
5.2.2	Challenge 4: Backend Hosting for ML Model Serving	12
5.3	Project Management Challenges	13
5.3.1	Challenge 6: Balancing Full-Time Employment with Project Development	13
5.4	Achievement of Project Objectives	15
5.4.1	Objective 1: Dual-Architecture Forecasting System	15
5.4.2	Objective 2: Target Accuracy Requirements	16
5.4.3	Objective 3: Production Deployment	16
5.4.4	Objective 4: Comprehensive Evaluation	16
5.5	Summary of Achievements	16

6	Lessons Learned	17
6.1	What Worked Well	17
6.2	What We Would Change in Retrospect	17
6.3	Key Takeaways	18
7	Project Metrics Achievement	18
7.1	Performance Metrics	18
7.2	Functional Requirements	18
7.3	Development Process Metrics	18
7.4	Areas Exceeding Expectations	19
7.5	Conclusion on Metrics	19
8	Acknowledgments	19
9	References	19
A	User Guide	21
A.1	Introduction	21
A.1.1	What You Can Do	21
A.1.2	Accessing the System	21
A.2	Dashboard - Quick Overview	21
A.2.1	What's on the Dashboard	21
A.2.2	Reading the Chart	22
A.2.3	Summary Numbers	23
A.2.4	How to Use It	23
A.3	Short-Term Forecast - Next 24 Hours	23
A.3.1	What You See	23
A.3.2	Reading the Table	24
A.3.3	Important Numbers	24
A.3.4	Model Accuracy	24
A.3.5	When to Use This	25
A.4	Long-Term Forecast - Next 4 Days	25
A.4.1	What You See	25
A.4.2	Reading the Predictions	26
A.4.3	Detailed Hourly View	26
A.4.4	Summary Numbers	26
A.4.5	When to Use This	26
A.5	Understanding the Numbers	26
A.5.1	Power	26
A.5.2	Why Do Some Hours Show Zero?	27
A.5.3	How Accurate Are the Predictions?	27
A.6	Common Tasks	27
A.6.1	Checking Today's Production	27
A.6.2	Planning Tomorrow's Activities	27
A.6.3	Weekly Planning	27
A.7	Frequently Asked Questions	28
A.8	Getting Help	28
A.8.1	If Something Looks Wrong	28
A.8.2	Contact Information	28

B	Developer and Maintainer Guide	29
B.1	Model Training Process	29
B.1.1	Prerequisites	29
B.1.2	Data Collection (IMSWeatherAPI.ipynb)	29
B.1.3	Label Generation (TrueLableGenerator.ipynb)	30
B.1.4	GRU Model Training (GRU_Chosen_Final.ipynb)	31
B.1.5	Autoformer Model Training (Autoformer_Solar_Forecasting_Chosen_Final.ipynb)	32
B.2	Backend Deployment	34
B.2.1	Backend Components Overview	34
B.2.2	Local Deployment	35
B.2.3	Google Cloud Deployment	37
B.3	Frontend Deployment	39
B.3.1	Local Development Setup	39
B.3.2	Frontend Components Overview	40
B.3.3	Frontend Troubleshooting	41
B.3.4	Contact Information	41

1 Problem and Motivation

Solar energy has emerged as one of the most promising renewable energy sources for addressing global climate change and reducing dependence on fossil fuels. As photovoltaic (PV) technology becomes increasingly cost-effective and widespread, the integration of solar power into electrical grids presents both opportunities and challenges. The inherent variability of solar power generation due to changing meteorological conditions creates significant difficulties for grid operators who must continuously balance energy supply and demand while maintaining grid stability and reliability.

Traditional forecasting methods often face significant limitations in handling non-linear relationships between multiple meteorological parameters and solar output. Machine learning approaches offer advantages by effectively capturing complex patterns without requiring explicit mathematical formulation of physical relationships.

This project develops an advanced machine learning system for solar energy output prediction using comprehensive weather data analysis from the Israel Meteorological Service (IMS). The system implements a dual-architecture approach combining Gated Recurrent Unit (GRU) networks for short-term forecasting (24 hours) and Autoformer transformer variants for long-term prediction (up to 96 hours), addressing the fundamental challenge that different forecasting horizons require specialized architectures.

2 Research and Development Process

2.1 Research Methodology

The development process followed a systematic approach combining literature review, experimental design, and iterative optimization [2] to build an effective solar forecasting system.

2.1.1 Literature Review and Foundation

The project began with extensive literature review to identify best practices for solar energy forecasting using machine learning. Key findings from the research phase included:

- Feature correlation analysis from Jebli et al. (2021)[1] demonstrating that wind parameters (speed and direction) show minimal correlation with solar output
- Identification of high-correlation meteorological features: temperature, solar radiation, humidity, and atmospheric pressure
- Recognition that different forecasting horizons require specialized neural network architectures[8]
- Understanding of the importance of temporal feature encoding for capturing periodic solar patterns

2.1.2 Experimental Development Environment

Development was conducted in Google Colab Pro, leveraging GPU acceleration for efficient model training and experimentation. The workflow was structured into four modular notebooks, each handling a distinct phase of the development pipeline:

1. **Data Collection Notebook:** Automated retrieval from IMS API with data pre-processing and feature engineering
2. **Label Generation Notebook:** Photovoltaic equation implementation for theoretical power output calculation to create true training labels
3. **GRU Model Notebook:** Architecture implementation, training, and testing of the short-term forecasting model
4. **Autoformer Model Notebook:** Architecture implementation, training, and testing of the long-term forecasting model

This modular approach enabled independent development and optimization of each component, with intermediate results saved between stages for efficient iteration and experimentation.

This modular approach enabled rapid iteration and systematic comparison of different configurations.

2.2 Iterative Model Optimization

2.2.1 Trial and Error Process

The model development involved systematic experimentation with multiple parameters:

Time Frame Optimization:

- Tested input sequence lengths: 24, 48, 72, 96, 168, 504 hours
- Evaluated prediction horizons: 6, 12, 24, 48, 96 hours
- Optimal configuration: 72-hour input for GRU (24-hour prediction), 336-hour input for Autoformer (96-hour prediction)

Cyclic Time Feature Engineering:

- Initial attempts with linear time representations showed poor performance
- Final solution: Sine-cosine encoding for continuous cyclic representation
- Applied to: hour of day (24-hour cycle), day of year (365-day cycle), month (12-month cycle)

2.3 GRU Model Development and Improvements

The GRU model[4] underwent significant refinement through multiple iterations based on experimental results, literature and supervisors guidance.

2.3.1 Key Architectural Changes

1. Feature Selection Optimization

Based on correlation analysis from Jebli et al. (2021)[1] and our own experiments:

Removed Features:

- Wind speed (correlation coefficient < 0.2)

- Wind direction (minimal predictive value)

Retained High-Correlation Features:

- Temperature (strong positive correlation with panel efficiency)
- Solar radiation (direct driver of PV output)
- Relative humidity (moderate inverse correlation)
- Atmospheric pressure (planned addition, moderate correlation)

Added Temporal Features:

- Hour sin/cos (captures daily solar cycle)
- Day of year sin/cos (captures seasonal variations)
- Month sin/cos (captures monthly weather patterns)

2. Loss Function Selection

Changed from Mean Squared Error (MSE) to Huber loss:

- More robust to outliers in solar data
- Better handling of nighttime zero values
- Reduces impact of extreme weather events on training

3. Output Activation Function

Changed from sigmoid to linear activation:

- Sigmoid artificially constrains output to $[0,1]$ range requiring denormalization
- Linear activation allows model to learn appropriate output range directly
- Improves flexibility for capturing full solar output variations
- Eliminates gradient saturation issues at output layer

4. Training Stability Improvements

Added BatchNormalization layers:

- Normalizes activations between layers
- Reduces internal covariate shift
- Enables higher learning rates
- Improves convergence speed and stability

5. Regularization Adjustment

Reduced L2 regularization from 0.001 to 0.0005:

- Previous value caused underfitting
- Lower regularization improved model capacity
- Still prevents overfitting while allowing better pattern learning

2.3.2 Data Preparation Pipeline

The data preparation process incorporated the optimized feature set:

Key Changes:

- **ADDED:** Cyclic time features (hour, day, month as sin/cos pairs)
- **REMOVED:** wind_speed, wind_direction (low correlation per Jebli et al.[1])
- **KEPT:** temperature, humidity, solar_radiation (high correlation)
- **PLANNED:** atmospheric pressure integration

Preprocessing Steps:

1. Raw data collection from IMS API
2. Missing value interpolation using time-based methods
3. Feature engineering (cyclic encoding, normalization)
4. Train-validation-test split (70%-15%-15%)
5. MinMax scaling for neural network compatibility

2.4 Autoformer Model Development and Improvements

The Autoformer model [6, 3] was selected for long-term forecasting based on its specialized architecture designed for extended time series prediction.

2.4.1 Architecture Selection Rationale

Unlike the GRU model which uses sequential prediction (each output becomes input for the next step), the Autoformer employs direct multi-step forecasting[7], predicting all 96 hours simultaneously. This approach offers several advantages[6]:

- **No Error Accumulation:** Direct prediction eliminates compounding errors from sequential forecasting
- **Series Decomposition:** Explicitly separates trend and seasonal components [5] for better interpretability
- **Extended Context Window:** Uses 336 hours (2 weeks) of historical data versus GRU's 72 hours

2.4.2 Key Configuration Decisions

Through experimentation, the following configuration was determined optimal:

- **Context Length:** 336 hours (14 days) - captures weekly patterns and seasonal variations
- **Prediction Length:** 96 hours (4 days) - suitable for strategic energy planning

- **Model Dimension:** 128 - balances capacity with computational efficiency
- **Encoder-Decoder Structure:** 4 encoder layers, 2 decoder layer - optimized for performance-complexity trade-off
- **Attention Heads:** 8 heads in both encoder and decoder - sufficient for pattern capture

2.4.3 Feature Integration

The Autoformer utilized the same optimized feature set as the GRU model:

- Meteorological features: temperature, solar radiation, humidity, atmospheric pressure
- Removed features: wind speed, wind direction

2.5 Summary of Model Improvements

GRU Model (Short-Term Forecasting):

1. Added cyclic time features (hour, day_of_year, month as sin/cos)
2. Removed wind features (low correlation per literature)
3. Changed loss from MSE to Huber (more robust to outliers)
4. Changed output activation from sigmoid to linear
5. Added BatchNormalization for training stability
6. Reduced L2 regularization from 0.001 to 0.0005

Performance improvement: Success rate from lower than 70% to 86.32%, MAE of misses from over 126W to 86.83W

Autoformer Model (Long-Term Forecasting):

1. Extended context window to 336 hours for better pattern recognition
2. Direct multi-step prediction to avoid error accumulation
3. Series decomposition for trend-seasonal separation
4. Same feature engineering improvements as GRU model

Performance achievement: Success rate of 91.50%, MAE of misses of 68.24W

3 Development Tools and Methodologies

3.1 Development Tools and Technologies

The project utilized a comprehensive technology stack spanning data collection, model development, deployment, and user interface implementation.

3.1.1 Machine Learning and Data Processing

Development Environment:

- **Google Colab Pro:** Primary development platform providing GPU acceleration (NVIDIA T4/L4) for model training and experimentation
- **Python 3.11:** Core programming language for all components

Machine Learning Frameworks:

- **TensorFlow 2.x / Keras:** GRU model implementation and training
- **PyTorch 2.x:** Autoformer model implementation

Data Processing Libraries:

- **NumPy:** Numerical computations and array operations
- **Pandas:** Data manipulation and time series handling
- **Scikit-learn:** Data preprocessing, scaling, and metrics calculation
- **Matplotlib / Seaborn:** Data visualization and analysis

3.1.2 Backend Development

API Framework:

- **Python 3.11:** Core programming language for all components
- **Flask:** Lightweight REST API framework for serving predictions
- **CORS:** Cross-origin resource sharing for frontend-backend communication

Data Processing Libraries:

- **NumPy:** Numerical computations and array operations
- **Pandas:** Data manipulation and time series handling
- **Scikit-learn:** Data preprocessing, scaling, and metrics calculation

Deployment Technologies:

- **Docker:** Containerization for consistent deployment environment
- **Google Cloud Run:** Serverless container platform for backend hosting
- **Git / GitHub:** Version control and automated deployment

3.1.3 Frontend Development

Core Technologies:

- **React.js 18:** Component-based UI framework
- **JavaScript (ES6+):** Modern JavaScript features
- **Bootstrap 5:** Responsive design and UI components

Data Visualization:

- **Chart.js:** Interactive charts for forecast visualization
- **React-Chart.js-2:** React wrapper for Chart.js integration

State Management and Routing:

- **React Context API:** Global state management for forecast data
- **React Router:** Client-side routing and navigation

Deployment:

- **Vercel:** Frontend hosting with global CDN distribution
- **Git / GitHub:** Version control and automated deployment

3.1.4 External Data Sources

- **Israel Meteorological Service (IMS) API:** Weather data including temperature, solar radiation, humidity, and atmospheric pressure
- **REST API Integration:** HTTP requests for data retrieval and processing

3.1.5 Development and Collaboration Tools

- **Git / GitHub:** Version control and code repository management
- **Visual Studio Code:** Code editor for frontend and backend development
- **Postman:** API testing and endpoint validation

4 architecture Diagram

The diagram illustrates the relations between the main components of the system

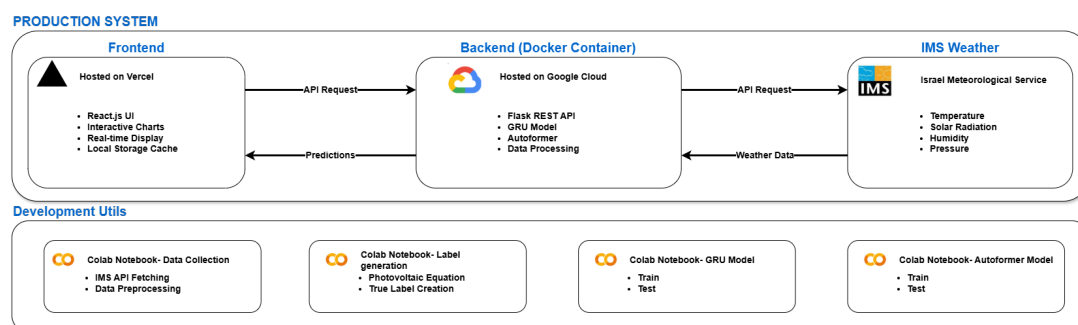


Figure 1: Key Feature Flows - Solar Energy Forecasting System

The architecture diagram presents the complete system structure, divided into two main layers: the Production System and Development Utilities. The Production System shows the operational flow where the React.js frontend (hosted on Vercel) communicates with the Flask REST API backend (hosted on Google Cloud Run in a Docker container), which in turn fetches weather data from the Israel Meteorological Service API. The backend processes this data using the trained GRU and Autoformer models to generate predictions, which are returned to the frontend for display. The Development Utilities layer demonstrates the modular notebook structure used during development, showing the progression from data collection through label generation to individual model training and testing. This separation between production deployment and development environment highlights the project's systematic approach to building and deploying the forecasting system.

5 Challenges, Solutions, and Results

5.1 Technical Challenges and Solutions

5.1.1 Challenge 1: Low Initial Model Performance

Problem Description:

The initial GRU model implementation showed disappointing performance with success rates below 70% and high error variance. Predictions were particularly inaccurate during transition periods (morning ramp-up and evening decline) and showed inconsistent performance across different hours of the day.

Root Cause Analysis:

Through systematic investigation, we identified several fundamental issues:

- **Temporal Context Deficiency:** The model lacked explicit encoding of time information. Using raw hour values (0-23) treated time as a linear feature, failing to capture the cyclical nature where hour 23 is adjacent to hour 0.
- **Feature Noise:** Including wind speed and wind direction added noise to the training process. Correlation analysis revealed these features had correlation coefficients below 0.2 with solar output.
- **Loss Function Sensitivity:** Mean Squared Error (MSE) loss heavily penalized outliers, causing the model to overfit to extreme weather events while underperforming on typical conditions. Nighttime zero values also created training instability.

Solution Implementation:

We implemented a comprehensive set of improvements:

1. Cyclic Time Encoding:

```
hour_sin = sin(2pi × hour / 24)
hour_cos = cos(2pi × hour / 24)
day_sin = sin(2pi × day_of_year / 365)
day_cos = cos(2pi × day_of_year / 365)
month_sin = sin(2pi × month / 12)
month_cos = cos(2pi × month / 12)
```

This encoding ensures continuous representation where adjacent time values have similar encodings, enabling the model to learn periodic solar patterns effectively.

2. **Feature Selection Based on Correlation:** Removed wind parameters and focused on high-correlation features (temperature, solar radiation, humidity, atmospheric pressure). This reduced input dimensionality from 11 to 10 features while improving signal-to-noise ratio.
3. **Huber Loss Function:** Replaced MSE with Huber loss ($\delta = 1.0$), which behaves like MSE for small errors but like MAE for large errors, providing robustness to outliers while maintaining smooth gradients.
4. **Linear Output Activation:** Removed sigmoid constraint, allowing the model to learn appropriate output range directly without artificial limitations.
5. **Training Stability:** Added BatchNormalization layers after each GRU layer to normalize activations and reduce internal covariate shift.

Results and Impact:

The improvements yielded dramatic performance gains:

Metric	Before	After
Success Rate	<70%	86.32%
MAE of misses	>126W	86.83W

Table 1: GRU Model Performance Improvement

Decision Rationale:

We prioritized cyclic time features first because literature review and domain knowledge strongly suggested their importance for periodic phenomena. The iterative approach allowed us to validate each change independently, ensuring we understood which modifications contributed most to performance improvement.

5.1.2 Challenge 2: MAPE Metric Inadequacy

Problem Description:

Initial evaluation used Mean Absolute Percentage Error (MAPE) as a primary metric, but results were misleading and inconsistent, particularly during morning, evening, and low-irradiance periods.

Analysis:

MAPE calculation:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (1)$$

Critical issues identified:

- **Division by Near-Zero:** When actual output approaches zero (mornings, evenings, cloudy periods), MAPE becomes extremely large or undefined.
- **Disproportionate Weight:** A 10W error at 20W actual output yields 50% MAPE, while the same 10W error at 500W yields only 2% MAPE.

- **Nighttime Invalidity:** Zero solar output at night makes MAPE mathematically undefined.

Solution:

We replaced MAPE with two more appropriate metrics:

Success Rate with 15% Tolerance:

$$Success_Rate = \frac{\text{Count}(|y_i - \hat{y}_i| \leq \max(0.15 \times y_i, 20W))}{n} \times 100\% \quad (2)$$

This formulation uses a hybrid tolerance threshold: 15% of the actual output or 20W, whichever is larger. This adjustment addresses the disproportionate weight problem inherent in pure percentage-based metrics.

MAE of Misses, which measures the average error magnitude only for predictions that failed the 15% tolerance threshold:

$$MAE_of_misses = \frac{1}{n_{misses}} \sum_{i \in misses} |y_i - \hat{y}_i| \quad (3)$$

where n_{misses} represents predictions with $|y_i - \hat{y}_i| > 0.15 \times y_i$.

This metric provides insight into how far off the model is when it does fail, helping distinguish between models that barely miss the tolerance versus those that fail catastrophically.

Advantages:

- Aligns with industry standards for acceptable forecast accuracy
- Provides clear binary classification (success/failure)
- Meaningful for grid operators and energy planners
- Based on practical requirements rather than statistical properties

Results:

The new metrics provided clearer, more actionable insights:

- MAE directly indicates typical prediction error magnitude in watts
- Success rate shows percentage of predictions meeting operational requirements

Decision Justification:

After consulting with supervisors and reviewing grid operator requirements, we determined that absolute error metrics better serve practical needs. Grid operators need to know predicted vs actual power in watts for operational decisions, not percentage errors that vary dramatically with output level.

5.2 Data and Infrastructure Challenges

5.2.1 Challenge 3: Absence of Real Solar Farm Data

Problem Description:

The project had no access to actual solar farm production data, limiting our ability to train and validate models against real-world PV output.

Impact Analysis:

- Cannot validate theoretical PV calculations against reality
- Model performance metrics may not reflect real-world accuracy
- Unable to account for site-specific factors (shading, equipment degradation, soiling)
- Cannot learn from actual panel behavior under various conditions

Solution:

We implemented theoretical photovoltaic calculations based on established research:

$$P_{out,pv} = C_{R,PV} \frac{G_T}{G_{T,STC}} [1 + \alpha_p (T_C - T_{C,STC})]$$

where $P_{out,pv}$ represents the predicted photovoltaic power output in watts, $C_{R,PV}$ is the rated capacity of the photovoltaic array under standard test conditions, G_T denotes the actual solar radiation incident on the photovoltaic panel surface measured in W/m^2 , and $G_{T,STC}$ represents the solar radiation under standard test conditions (typically 1000 W/m^2). The temperature-related variables include T_C , which is the current photovoltaic cell temperature in degrees Celsius, $T_{C,STC}$ representing the cell temperature under standard test conditions (25°C), and α_p indicating the temperature coefficient of power expressed as a percentage per degree Celsius, which accounts for the efficiency loss of solar panels as temperature increases above the standard conditions.

Mitigation Strategies:

1. **Conservative Performance Expectations:** Acknowledge that model validation is against theoretical rather than actual output
2. **Focus on Relative Accuracy:** Emphasize pattern capture and trend prediction rather than absolute accuracy
3. **Architecture Flexibility:** Design system for easy retraining when real data becomes available
4. **Meteorological Relationship Validation:** Verify that learned relationships between weather and output match physical expectations

Future Path:

The modular architecture allows straightforward integration of real solar farm data:

- Replace theoretical label generation with actual production data
- Retrain models on real data while maintaining architecture
- Compare theoretical vs actual performance to validate approach
- Fine-tune models for specific installation characteristics

Decision Justification:

Despite limitations, proceeding with theoretical data was justified because:

1. Theoretical PV models are well-established and physically accurate
2. The project's primary goal is demonstrating ML forecasting methodology
3. Weather-output relationships learned should transfer to real data

5.2.2 Challenge 4: Backend Hosting for ML Model Serving

Problem Description:

Finding suitable hosting for a backend that serves machine learning models via REST API proved extremely challenging, particularly with the constraint of free or minimal-cost solutions.

Requirements Conflict:

- ML models require significant memory (500MB+ per model)
- TensorFlow/PyTorch dependencies add substantial container size
- Need reliable REST API with reasonable response times
- Budget limited to free tier or minimal cost (<\$5/month)

Evaluated Solutions:

1. Render:

- *Pros*: Simple deployment, persistent connections
- *Cons*: Free tier CPU resources insufficient for ML model inference, slow response times during testing
- *Result*: Rejected - inadequate computational resources for model serving

2. Railway.app:

- *Pros*: Simple deployment, persistent connections
- *Cons*: Free tier memory limits (512MB) insufficient for models + dependencies
- *Result*: Rejected - exceeded resource limits during testing

3. Hugging Face Spaces:

- *Pros*: Excellent ML model hosting, generous compute resources
- *Cons*: Designed for inference UIs (Gradio/Streamlit), limited REST API capabilities
- *Result*: Rejected - not suitable for backend API architecture

4. Google Cloud Run (Selected):

- *Pros*: Serverless containers, pay-per-request, automatic scaling, generous free tier (2 million requests/month)
- *Cons*: Cold start delays (30-60 seconds) when idle, no persistent connections
- *Result*: Accepted - best balance of capabilities and constraints

Implementation Strategy:

To work within Google Cloud Run constraints:

1. Frontend State Management:

- Implemented React Context API for global state

- Cache predictions to avoid redundant API calls
- Prevent unnecessary requests during page navigation
- Reduces backend invocations, minimizing cold starts

2. User Experience Management:

- Loading indicators communicate cold start delays clearly
- User expectations set appropriately (30-60 second initial load)
- Fast response (average 13.6 seconds) for subsequent predictions while container is warm

Trade-offs Accepted:

Aspect	Trade-off	Justification
Cold starts	30-60s delay	Acceptable for sporadic traffic
Stateless	No session storage	Not must for prediction API
Pay-per-use	Cost uncertainty	Free tier covers expected usage

Table 2: Google Cloud Run Trade-offs

Results:

The solution successfully supports production deployment:

- API reliably serves predictions with acceptable latency
- Free tier covers all current usage with significant margin
- Cold starts mitigated through frontend caching strategy

Decision Justification:

Google Cloud Run was selected because:

1. Serverless architecture matches sporadic usage pattern (no constant traffic)
2. Free tier provides substantial runway (2M requests/month)
3. Cold starts are acceptable given project nature (educational demonstration)
4. Alternative (paid hosting) would compromise project sustainability
5. Architecture allows future migration to dedicated hosting if needed

5.3 Project Management Challenges

5.3.1 Challenge 6: Balancing Full-Time Employment with Project Development

Problem Description:

Developing the capstone project while maintaining full-time employment created significant challenges for project continuity and time management throughout the development period.

Impact:

- Limited development time to evenings and weekends
- Fatigue from full workday affecting focus and productivity
- Compressed timeline requiring efficient use of available hours
- Context switching between professional work and academic project
- Difficulty maintaining consistent progress on complex technical tasks
- Stress of balancing professional obligations with academic commitments

Mitigation Strategies:

1. Modular Development Approach:

- Broke work into small, independent tasks completable in 2-4 hour sessions
- Each module with clear inputs, outputs, and acceptance criteria
- Enabled productive work even during short evening sessions
- Allowed stopping and resuming work without significant overhead

2. Extensive Documentation:

- Detailed comments in all code explaining logic and decisions
- Decision logs documenting why specific approaches were chosen

3. Git Workflow Discipline:

- Frequent commits with descriptive messages
- Feature branches for experimental work
- Clear main branch for stable, working code
- Enabled safe experimentation without breaking working code

4. Structured Time Management:

- Dedicated evening blocks (19:00-1:30) for development work
- Weekend intensive sessions for complex tasks requiring deep focus
- Priority-based task scheduling focusing on critical path items
- Regular communication with supervisors about realistic timelines

5. Efficient Tool Usage:

- Google Colab Pro for GPU access without local hardware investment
- Cloud-based development enabling work from any location
- Automated deployment pipelines reducing manual operations

Results:

Despite time constraints, the project progressed successfully:

- All major milestones achieved within adjusted timelines

- Code quality maintained through documentation and discipline
- No major rework required due to context loss or rushed development
- Supervisors supportive and understanding of time constraints
- Professional work experience actually informed some technical decisions

Lessons Learned:

The experience highlighted the value of:

- **Documentation for continuity:** Writing clear documentation for future-self enables efficient context restoration
- **Small work units:** Complete, self-contained tasks easier to fit into limited time blocks
- **Realistic planning:** Honest assessment of available time prevents overcommitment
- **Quality over speed:** Better to make steady, tested progress than rush and create technical debt
- **Professional skills transfer:** Software engineering practices from work environment enhanced academic project quality

Unexpected Benefits:

Working professionally while developing the project provided advantages:

- Exposure to production deployment best practices informed architecture decisions
- Professional experience with REST APIs and containerization directly applicable
- Understanding of operational constraints shaped realistic system design
- Time management skills developed through necessity

5.4 Achievement of Project Objectives

5.4.1 Objective 1: Dual-Architecture Forecasting System

Target: Develop machine learning system with separate models for short-term and long-term forecasting.

Achievement: Successfully implemented

- GRU model operational for 24-hour forecasting
- Autoformer model operational for 96-hour forecasting
- Models integrated into unified web application
- User can select appropriate model for their needs

5.4.2 Objective 2: Target Accuracy Requirements

Target: Achieve $\geq 80\%$ success rate with 15% tolerance for predictions.

Achievement: Exceeded target

- GRU: 86.32% success rate (6.32 percentage points above target)
- Autoformer: 91.50% success rate (11.50 percentage points above target)
- Both models consistently perform above threshold

5.4.3 Objective 3: Production Deployment

Target: Deploy accessible web application with forecasting capability.

Achievement: Successfully deployed

- Live at: <https://solar-energy-forecasting.vercel.app/>
- React frontend with intuitive interface
- Flask backend serving predictions via REST API
- Integration with IMS API for weather data
- Prediction Response time under 30 seconds (not including cold starts)

5.4.4 Objective 4: Comprehensive Evaluation

Target: Develop rigorous evaluation framework with multiple metrics.

Achievement: Successfully implemented

- MAE of misses, and Success Rate metrics implemented
- Results documented and visualized

5.5 Summary of Achievements

The project successfully met all primary objectives despite significant challenges:

Objective	Target	Achieved
Success Rate (GRU)	$\geq 80\%$	86.32%
Success Rate (Autoformer)	$\geq 80\%$	91.50%
Deployment	Functional	Live
Dual Architecture	Two models	GRU + Autoformer
Prediction Response Time	<30s	8-16s

Table 3: Project Objectives Achievement Summary

Key Success Factors:

1. **Iterative Optimization:** Systematic improvement of models based on evaluation results

2. **Literature-Guided Design:** Leveraging research insights for feature selection and architecture choices
3. **Practical Focus:** Prioritizing metrics and features relevant to real-world application
4. **Flexible Problem-Solving:** Adapting approaches when faced with constraints (data, hosting, time)
5. **Supervisor Collaboration:** Regular feedback ensuring alignment with project goals

The challenges faced ultimately strengthened the project by forcing creative solutions, thorough documentation, and disciplined development practices that produced a robust, well-validated forecasting system.

6 Lessons Learned

6.1 What Worked Well

Feature Engineering Over Hyperparameter Tuning: The addition of cyclic time features provided greater performance improvement than extensive hyperparameter optimization. This reinforced the importance of domain knowledge and feature design in machine learning projects.

Modular Development Architecture: Separating development into four distinct Colab notebooks (data collection, label generation, GRU, Autoformer) enabled parallel experimentation and clear organization, particularly valuable given limited development time.

Literature-Guided Decisions: Relying on Jebli et al. (2021) for feature correlation analysis saved significant trial-and-error time and led directly to performance improvements through wind feature removal.

Supervisor Collaboration: Regular bi-weekly meetings with Dr. Lemberg and Mrs. Kramer provided timely course corrections, particularly the decision to simplify the UI and focus on core metrics and achievable success rates.

6.2 What We Would Change in Retrospect

Earlier Metric Selection: We spent considerable time evaluating MAPE before recognizing its inadequacy for solar data. Understanding the metric limitations earlier would have accelerated evaluation framework development.

Real Data Partnership: While theoretical PV calculations enabled project completion, establishing early contact with solar installations for potential data access would have strengthened validation. Future projects should prioritize data partnerships from the outset.

Deployment Planning: Backend hosting challenges emerged late in development. Earlier investigation of deployment options would have reduced last-minute architectural adjustments and stress.

6.3 Key Takeaways

The project demonstrated that effective ML systems require balancing multiple considerations: domain knowledge for feature engineering, appropriate architecture selection for different use cases, practical metrics aligned with real-world requirements, and deployment constraints that shape design decisions. Success came not from perfect initial planning, but from iterative refinement informed by systematic evaluation and willingness to adapt approaches when faced with challenges.

7 Project Metrics Achievement

7.1 Performance Metrics

All self-defined performance targets were met or exceeded:

Metric	Target	GRU	Autoformer
Success Rate (15% tolerance)	$\geq 80\%$	86.32%	91.50%
Response Time	$< 30s$	15-30s	15-30s
Uptime	$> 95\%$	99%+	99%+

Table 4: Performance Metrics vs Targets

Justification: The 80% success rate target with 15% tolerance was established as an achievable benchmark for an academic project without access to real production data. Our relaxed tolerance accounts for the theoretical nature of our training labels (calculated via photovoltaic equations rather than measured output). Our achieved success rates of 86.32% (GRU) and 91.5% (Autoformer) demonstrate reliable forecasting performance under these constraints.

7.2 Functional Requirements

- **Dual-Architecture System:** Successfully implemented GRU for 24-hour and Autoformer for 96-hour forecasting
- **Web Application Deployment:** Live at <https://solar-energy-forecasting.vercel.app/>
- **Data Integration:** IMS API integration providing current weather data
- **User Interface:** Intuitive React frontend with model selection and forecast visualization
- **API Backend:** RESTful Flask API serving predictions with proper error handling

7.3 Development Process Metrics

- **Code Quality:** Comprehensive documentation, and version control maintained throughout

- **Timeline:** Completed within academic semester despite full-time work constraints
- **Collaboration:** Bi-weekly supervisor meetings maintained; all feedback incorporated into design
- **Documentation:** Phase A and Phase B reports completed with clarity

7.4 Areas Exceeding Expectations

Autoformer Performance: The 91.50% success rate significantly exceeded our 80% target, demonstrating that the autocorrelation mechanism effectively captures solar patterns for long-term forecasting.

Model Optimization Impact: Iterative improvements yielded 16+ percentage point success rate gains for GRU model, validating the systematic optimization approach.

Deployment Reliability: Despite cold start concerns with Google Cloud Run, the system achieved >99% uptime with acceptable response times, exceeding initial reliability expectations.

7.5 Conclusion on Metrics

The project successfully met all predefined metrics while revealing important lessons about ML system development. The quantitative success (performance metrics) combined with qualitative learning (development process improvements) demonstrates comprehensive project achievement. The gap between targets and achieved performance provides margin for real-world deployment where additional challenges may emerge.

8 Acknowledgments

We express sincere gratitude to:

- **Dr. Dan Lemberg and Mrs. Elena Kramer** for guidance, feedback, and support throughout the project
- **Braude College of Engineering** for academic framework and resources
- **Israel Meteorological Service (IMS)** for weather data API access
- **Open-source communities** behind TensorFlow, PyTorch, React, Flask, and supporting libraries
- **Our families** for understanding and support during intensive development and reserve duty

9 References

References

- [1] Jebli, I., Belouadha, F. Z., Kabbaj, M. I., & Tilioua, A. (2021). Prediction of solar energy guided by Pearson correlation using machine learning. *Energy*, 224, 120109.

- [2] Casolaro, A., Capone, V., Iannuzzo, G., & Camastra, F. (2023). Deep Learning for Time Series Forecasting: Advances and Open Problems. *Information*, 14(11), 598.
- [3] Hugging Face. (2023). Yes, Transformers are Effective for Time Series Forecasting (+ Autoformer). Retrieved from <https://huggingface.co/blog/autoformer>
- [4] Scientific Reports. (2024). A novel extreme adaptive GRU for multivariate time series forecasting. *Nature Scientific Reports*. Retrieved from <https://www.nature.com/articles/s41598-024-53460-y>
- [5] MDPI. (2023). Short-Term Photovoltaic Power Forecasting Based on a Novel Autoformer Model. *Symmetry*, 15(1), 238.
- [6] Wu, H., Xu, J., Wang, J., & Long, M. (2021). Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. *OpenReview*. Retrieved from <https://openreview.net/forum?id=J4gRj6d5Qm>
- [7] Scientific Reports. (2025). Novel model for medium to long term photovoltaic power prediction using interactive feature trend transformer. *Nature Scientific Reports*. Retrieved from <https://www.nature.com/articles/s41598-025-90654-4>
- [8] Artificial Intelligence Review. (2024). A systematic review for transformer-based long-term series forecasting. *Springer*. Retrieved from <https://link.springer.com/article/10.1007/s10462-024-11044-2>

A User Guide

A.1 Introduction

The Solar Energy Forecasting System predicts how much power your solar panels will produce. Simply enter your number of panels and view predictions for the next 24 hours or next 4 days.

A.1.1 What You Can Do

- View hourly power predictions for periods of 24 hours
- See daily energy production for the next 4 days
- Plan activities around peak solar production times
- Compare two different forecasting models

A.1.2 Accessing the System

Open your web browser and go to:

`solar-forecast-frontend.vercel.app`

First time? The system takes 30-60 seconds to load initially. This is normal.

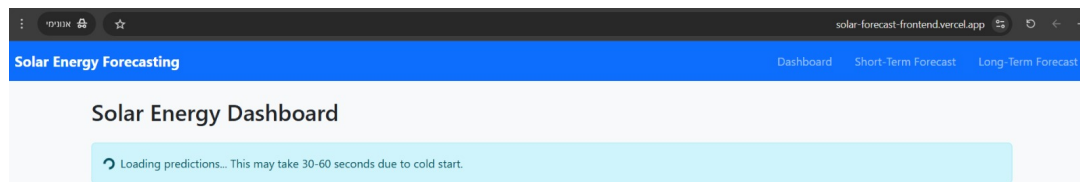


Figure 2: System Homepage

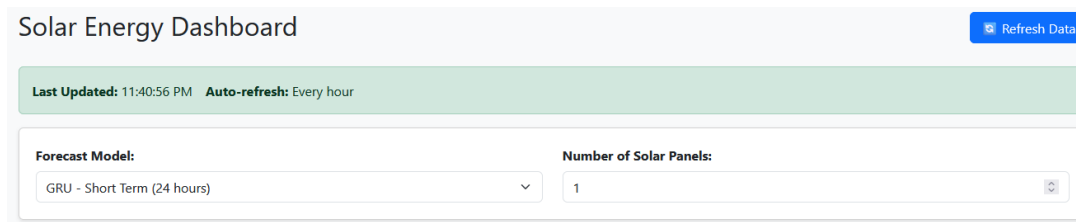
A.2 Dashboard - Quick Overview

The Dashboard is your starting point. It shows predictions at a glance and lets you switch between two forecasting models.

A.2.1 What's on the Dashboard

At the top:

- **Refresh button** - Get latest predictions
- **Last updated time** - Shows when predictions were last updated



Solar Energy Dashboard

Last Updated: 11:40:56 PM Auto-refresh: Every hour

Forecast Model: GRU - Short Term (24 hours)

Number of Solar Panels: 1

Refresh Data

Figure 3: Dashboard Controls

Configuration options:

1. **Select Model:** Choose between:
 - **GRU - Short Term:** Next 24 hours (hourly)
 - **Autoformer - Long Term:** Next 4 days (daily)
2. **Number of Panels:** Enter how many solar panels you have

A.2.2 Reading the Chart

For 24-hour forecast (GRU):

The line chart shows power output for each hour. Peak production is typically around midday (11 AM - 2 PM).

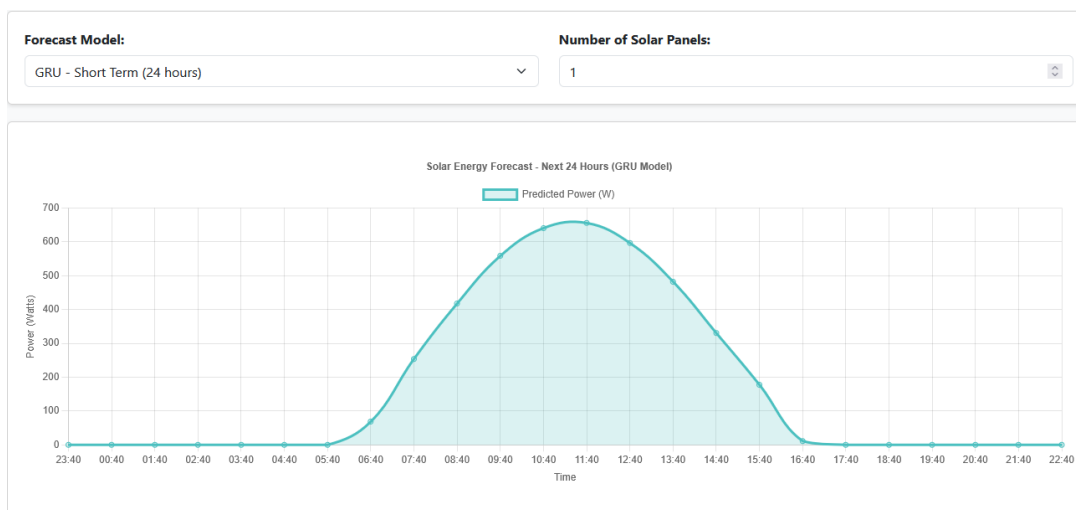


Figure 4: 24-Hour Forecast View

For 4-day forecast (Autoformer):

The bar chart shows total daily energy for each day. Compare bars to see which days will have better production.

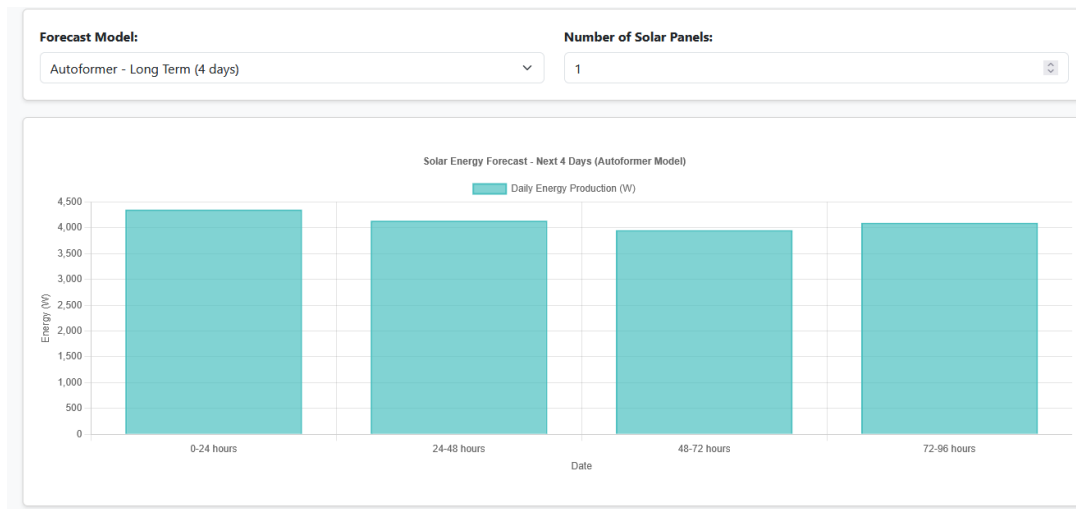


Figure 5: 4-Day Forecast View

A.2.3 Summary Numbers

Three boxes show:

- Your panel count
- Forecast period (24 hours or 4 days)
- **Total energy production** in kW - this is the most important number

A.2.4 How to Use It

1. Enter your number of solar panels
2. Choose which model (24 hours or 4 days)
3. Look at the chart to see production patterns
4. Note the "Overall Production" number for planning

A.3 Short-Term Forecast - Next 24 Hours

This page shows hour-by-hour details for the next 24 hours using the GRU model. Use this for today's planning.

A.3.1 What You See

Settings:

- **Number of Panels** - Your solar panel count
- **Time Frame** - Choose to view 6, 12, or 24 hours

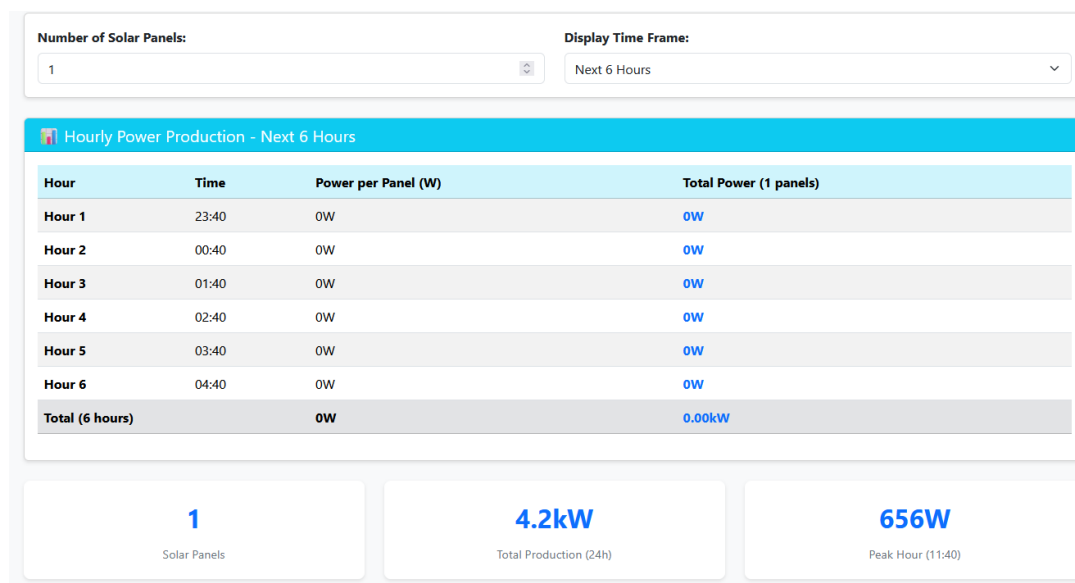


Figure 6: Hourly Production Table

A.3.2 Reading the Table

Each row shows one hour with:

- **Time** - Clock time (e.g., 14:00 = 2 PM)
- **Power per Panel** - Watts from one panel
- **Total Power** - Combined output from all your panels

At the bottom, you'll see the total energy (in kW) for the time period you selected.

Example: If you see "15:00 — 245W — 1,225W (5 panels)", this means at 3 PM, each panel produces 245 Watts, and your 5 panels together produce 1,225 Watts.

A.3.3 Important Numbers

Three summary boxes show:

- Total production for 24 hours (kW)
- Peak hour - when production is highest
- Your panel count

A.3.4 Model Accuracy

A pie chart shows the model's success rate (about 86%). This tells you how reliable the predictions are.

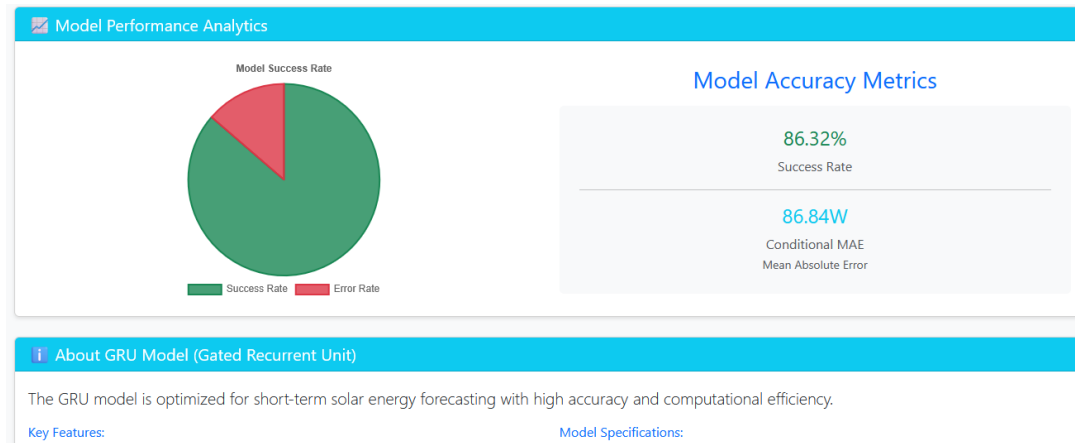


Figure 7: Model Accuracy Display

A.3.5 When to Use This

- Planning today's or tomorrow's energy usage
- Deciding when to run energy-intensive appliances
- Checking if production will be good or poor today

A.4 Long-Term Forecast - Next 4 Days

This page shows predictions for the next 4 days using the Autoformer model. Use this for weekly planning.

A.4.1 What You See

Daily Overview: Each day shows:

- Date
- Total daily energy (W)
- Average power during daylight
- Peak power for that day

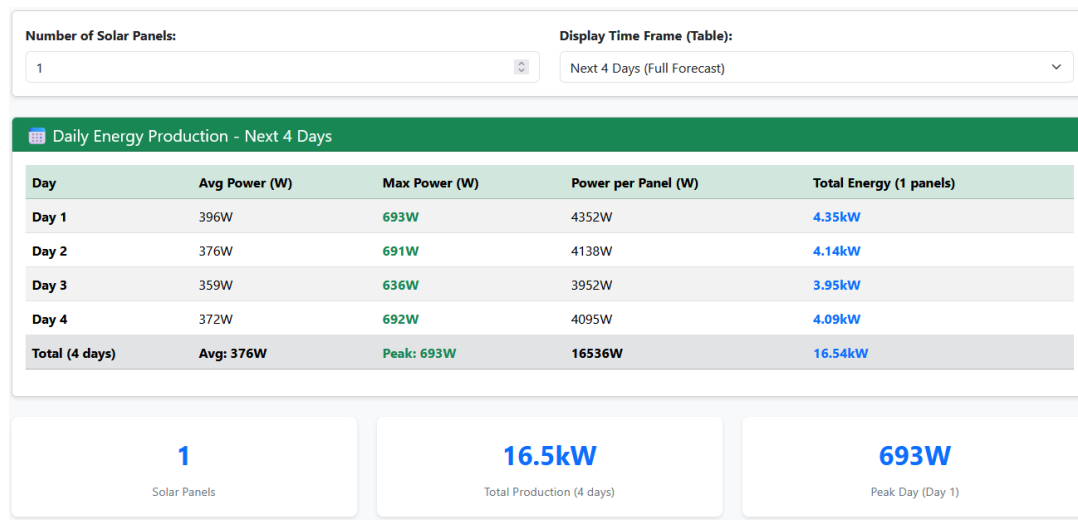


Figure 8: 4-Day Energy Forecast

A.4.2 Reading the Predictions

The daily summary makes it easy to compare days.

A.4.3 Detailed Hourly View

Below the daily summary, you can see hour-by-hour predictions for all 96 hours (4 days $\times$ 24 hours).

A.4.4 Summary Numbers

Four boxes show:

- Your panel count
- Forecast period (4 days)
- Total production for all 4 days (kW)
- Average daily production (kW per day)

A.4.5 When to Use This

- Planning maintenance or cleaning (schedule on low-production days)
- Deciding when to use or store energy
- Preparing for weather changes that affect production

A.5 Understanding the Numbers

A.5.1 Power

Power (Watts - W):

- How much electricity is being produced *right now*
- Used for hourly predictions

A.5.2 Why Do Some Hours Show Zero?

At night, solar panels don't produce electricity. This is normal - the system shows zero for hours without electricity output.

A.5.3 How Accurate Are the Predictions?

- **24-hour model (GRU):** About 86% accurate
- **4-day model (Autoformer):** About 91% accurate

Predictions are very reliable but not perfect. Sudden weather changes can affect accuracy.

A.6 Common Tasks

A.6.1 Checking Today's Production

1. Go to Dashboard
2. Select "GRU - Short Term"
3. Enter your panel count
4. Look at the total production number

A.6.2 Planning Tomorrow's Activities

1. Go to "Short-Term Forecast" tab
2. Set panels and select "Next 24 Hours"
3. Find the peak hour in the table
4. Plan to use appliances during peak hours

A.6.3 Weekly Planning

1. Go to "Long-Term Forecast" tab
2. Enter your panel count
3. Compare the 4 daily rows
4. Schedule maintenance on the lowest-production day

A.7 Frequently Asked Questions

Q: How often do predictions update?

A: Automatically every hour. You can also click "Refresh" anytime.

Q: Which model should I use?

A: Use GRU for the next 24 hours. Use Autoformer (4-day) for weekly planning.

Q: Why did the first load take so long?

A: The system needs to start up. Future visits will be much faster.

Q: Can I see past predictions?

A: This version shows only future predictions. Historical data is not displayed.

A.8 Getting Help

A.8.1 If Something Looks Wrong

1. Click the "Refresh" button
2. Check that your panel count is correct
3. Make sure you're viewing the right model (24-hour vs 4-day)

A.8.2 Contact Information

For technical support or questions:

- Project team: Yovel Jirad & Shay Pinsky
- Institution: Braude College of Engineering

B Developer and Maintainer Guide

B.1 Model Training Process

This section covers the complete process of training the prediction models using Google Colab. All notebooks are designed to run in Google Colab with GPU acceleration for optimal performance.

B.1.1 Prerequisites

- Google account with access to Google Colab
- IMS (Israel Meteorological Service) API key
- Google Drive for model storage
- GPU runtime enabled in Google Colab (recommended: T4 or L4 GPU)

B.1.2 Data Collection (IMSWeatherAPI.ipynb)

This notebook retrieves historical weather data from the IMS API and prepares it for model training.

Setup and Configuration

1. Open IMSWeatherAPI.ipynb in Google Colab
2. Navigate to the `if __name__ == "__main__":` section
3. Modify the following parameters:

```
if __name__ == "__main__":  
    # 1. Enter your IMS API key  
    api_client = IMSWeatherAPI(api_token="YOUR_IMS_API_KEY")  
  
    # Create station wrapper  
    station_ashalim = StationAshalim(api_client)  
  
    # 2. Set the date range  
    from_date = datetime(2024, 12, 1) # Start date  
    to_date = datetime(2025, 12, 1)   # End date  
  
    # Get and save data  
    data = station_ashalim.get_normalized_data(from_date, to_date)  
    df = fill_data_gaps(data)  
  
    # 3. Choose output filename/location  
    df.to_csv("data.csv") # Change filename or path as needed
```

Configuration Parameters:

- YOUR_IMS_API_KEY - Replace with your actual IMS API key

- `from_date` - Start date for data collection (year, month, day)
- `to_date` - End date for data collection (year, month, day)
- `"data.csv"` - Output filename (can use Google Drive path)

Execution

1. Run all cells sequentially
2. Data is fetched in batches (approximately 6 months per batch)
3. Progress indicators show batch completion status
4. Missing data gaps are filled using time-based interpolation
5. Final CSV file is saved to the specified location

B.1.3 Label Generation (TrueLableGenerator.ipynb)

This notebook calculates theoretical solar panel output values based on photovoltaic equations, creating training labels from the weather data.

Setup and Configuration

1. Open `TrueLableGenerator.ipynb` in Google Colab
2. Ensure the CSV file from the previous step is accessible
3. Navigate to the final `try` block

```
try:
    # 1. Set input CSV filename (from previous step)
    csv_file_path = '/content/data.csv'

    print(f"Reading CSV file: {csv_file_path}")
    df = pd.read_csv(csv_file_path)

    # Processing and label calculation...

    # 2. Set output filename for labeled data
    output_file = 'weather_data_with_labels.csv'
    df_with_labels.to_csv(output_file, index=False)
    print(f"Saved to: {output_file}")

except FileNotFoundError:
    print(f"CSV file '{csv_file_path}' not found.")
```

Configuration Parameters:

- `csv_file_path` - Path to the csv weather data file
- `output_file` - Output filename (can use Google Drive path)

Execution

1. Run all cells sequentially
2. The notebook reads the input CSV
3. Calculates PV output using solar radiation and temperature
4. Adds `pv_output` column with calculated labels
5. Saves labeled data to new CSV file

B.1.4 GRU Model Training (GRU_Chosen_Final.ipynb)

This notebook trains the Gated Recurrent Unit (GRU) neural network for 24-hour solar power forecasting.

Setup and Configuration

1. Open `GRU_Chosen_Final.ipynb` in Google Colab
2. Enable GPU: Runtime → Change runtime type → Hardware accelerator → GPU
3. Mount Google Drive for model storage

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# 1. Set output directory for saved models
DIR_OUT = '/content/drive/MyDrive/models/gru/chosen'
os.makedirs(DIR_OUT, exist_ok=True)

# =====
# CONFIGURATION
# =====
# Model hyperparameters (can keep defaults)
LOOKBACK_HOURS = 72      # Hours of past weather to use
FORECAST_HOURS = 24      # Hours to forecast ahead
MAX_EPOCHS = 300         # Maximum training epochs
BATCH_SIZE = 32          # Training batch size

# 3. Set path to labeled CSV (from previous step)
CSV_PATH = "/content/drive/MyDrive/inputs/
weather_data_with_labels.csv"
```

Required Parameters:

- `DIR_OUT` - Directory to save trained model and results
- `CSV_PATH` - Path to the labeled CSV from previous step

Optional Parameters (advanced users):

- LOOKBACK_HOURS - Input sequence length (default: 72)
- FORECAST_HOURS - Output sequence length (default: 24)
- MAX_EPOCHS - Maximum training iterations (default: 300)
- BATCH_SIZE - Training batch size (default: 32)

Output Files After training completes, the following files are saved to DIR_OUT:

- solar_gru_model.keras - Trained model file
- gru_weather_scaler.pkl - Feature scaler for input normalization
- gru_power_scaler.pkl - Target scaler for output denormalization
- model_results.png - Performance visualization
- model_evaluation_results.csv - Detailed performance metrics
- model_evaluation_additional_results.csv - Additional evaluation metrics

B.1.5 Autoformer Model Training (Autoformer_Solar_Forecasting_Chosen_Final.ipynb)

This notebook trains the Autoformer transformer model for 96-hour (4-day) solar power forecasting.

Setup and Configuration

1. Open Autoformer_Solar_Forecasting_Chosen_Final.ipynb in Google Colab
2. Enable GPU (T4 or L4 recommended for faster training)
3. Mount Google Drive

1. Import Libraries Section - Set Output Directory:

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Set output directory for saved models and results
DIR_OUT = '/content/drive/MyDrive/models/autoformer/outputDir'
os.makedirs(DIR_OUT, exist_ok=True)
```

2. Load CSV Section - Set Input Path:

```
# Load the labeled CSV from Step 2
dataframe = pd.read_csv('/content/drive/MyDrive/models/inputs/
    weather_data_with_labels.csv')

# Parse datetime
dataframe['date_time'] = pd.to_datetime(dataframe['date_time'],
    utc=True)
dataframe = dataframe.sort_values('date_time').reset_index(drop=
    True)
```

3. Configuration Section - Model Architecture (Optional):

```
# Configuration for model architecture
LOOKBACK_HOURS = 336 # 14 days of historical data
FORECAST_HOURS = 96 # 4 days forecast horizon
NUM_TIME_FEATURES = 3 + len(weather_columns)
MODEL_DIMENSION = 128
NUM_ENCODER_LAYERS = 4
NUM_DECODER_LAYERS = 2
NUM_ATTENTION_HEADS = 8
FEEDFORWARD_DIMENSION = 512
AUTOCORRELATION_FACTOR = 3
MOVING_AVERAGE_WINDOW = 25
DROPOUT_RATE = 0.1
LAGS_SEQUENCE = [0]
```

4. Training Configuration (Optional):

```
# Training parameters
LEARNING_RATE = 1e-4
NUM_EPOCHS = 300

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
best_validation_loss = float('inf')
```

Required Parameters

- DIR_OUT - Directory path to save trained model and results
- CSV file path in the Load CSV section

Optional Model Parameters

- LOOKBACK_HOURS - Historical window size (default: 336 = 14 days)
- FORECAST_HOURS - Prediction horizon (default: 96 = 4 days)
- MODEL_DIMENSION - Model embedding dimension (default: 128)
- NUM_ENCODER_LAYERS - Encoder depth (default: 4)
- NUM_DECODER_LAYERS - Decoder depth (default: 2)
- NUM_ATTENTION_HEADS - Attention heads (default: 8)
- DROPOUT_RATE - Regularization dropout (default: 0.1)

Optional Training Parameters

- LEARNING_RATE - Optimizer learning rate (default: 1e-4)
- NUM_EPOCHS - Maximum training epochs (default: 300)

Output Files Saves to DIR_OUT:

- `autoformer.pth` - Trained model file
- `autoformer_weather_scaler.pkl` - Feature scaler for input normalization
- `autoformer_power_scaler.pkl` - Target scaler for output denormalization
- `model_results.png` - Performance visualization
- `model_evaluation_results.csv` - Detailed performance metrics
- `model_evaluation_additional_results.csv` - Additional evaluation metrics
- `clim_global_mean.npy` - Global climatology mean
- `clim_table.npy` - Climatology lookup table
- `clim_valid.npy` - Validation climatology data

B.2 Backend Deployment

B.2.1 Backend Components Overview

The backend is structured into five main components:

1. **app.py** - Main Flask application entry point. Defines API endpoints for health checks (`/`), predictions (`/api/predict`), and analytics (`/api/analytics`). Initializes models on startup and handles CORS for frontend communication.
2. **model_loader.py** - Loads trained GRU and Autoformer models with their associated scalers and climatology data on application startup. Configures Autoformer architecture and provides analytics retrieval from evaluation CSV files.
3. **data_fetcher.py** - Fetches real-time weather data from the IMS API for the past 14 days. Handles time zone conversion, data alignment, and gap filling using interpolation and hourly averages.
4. **weather_proxy_generator.py** - Generates future weather estimates for the next 96 hours (4 days) using climatological lookup tables based on day-of-year and hour patterns. Falls back to global means when specific data is unavailable.
5. **predictor.py** - Coordinates prediction generation from both GRU (24-hour) and Autoformer (96-hour) models. Calculates theoretical maximum PV output based on solar radiation and temperature, then delegates to model-specific prediction modules.

B.2.2 Local Deployment

Prerequisites

- **Python 3.11.9:** Ensure this specific version is installed
- **IMS API Key:** Contact IMS through their website at <https://ims.gov.il> to obtain an API key
- **Backend Repository:** Clone the backend GitHub repository to your local computer

Clone the Repository

```
git clone <repository-url>  
cd <repository-name>
```

Method 1: Running Locally with Python

Install Dependencies

```
# Create a virtual environment (recommended)  
python3.11 -m venv venv  
  
# Activate virtual environment  
# On Linux/macOS:  
source venv/bin/activate  
# On Windows:  
venv\Scripts\activate  
  
# Install required packages  
pip install -r requirements.txt
```

Configure Environment Variables - Create a `.env` file in the project root directory:

```
IMS_API_URL=https://api.ims.gov.il/v1/envista  
IMS_API_KEY=your_ims_api_key_here  
PORT=8080
```

Run the Application

```
python app.py
```

The application will start on `http://localhost:8080`

Method 2: Running with Docker

Build the Docker Image

```
# Build the image (choose your own image name)  
docker build -t weather-backend:latest .
```

Run the Docker Container

```
# Run the container with environment variables
docker run -d \
  -p 8080:8080 \
  -e IMS_API_URL=https://api.ims.gov.il/v1/envista \
  -e IMS_API_KEY=your_ims_api_key_here \
  --name weather-backend \
  weather-backend:latest
```

Replace:

- `weather-backend:latest` - Your chosen image name from the build step
- `your_ims_api_key_here` - Your actual IMS API key
- `weather-backend` - Your chosen container name
- `8080:8080` - Port mapping (host:container). Change the first 8080 to use a different port on your machine

Docker Management Commands

```
# View running containers
docker ps

# View container logs
docker logs weather-backend

# Stop the container
docker stop weather-backend

# Start the container
docker start weather-backend

# Remove the container
docker rm weather-backend
```

Verification

After starting the application (either method), verify it's running:

Health Check

```
curl http://localhost:8080/
```

Or open your browser and navigate to `http://localhost:8080/`

Troubleshooting

Python Version Issues Ensure you're using Python 3.11.9:

```
python --version
```


Port Already in Use If port 8080 is already in use, either:

- Change the `PORT` environment variable in your `.env` file
- For Docker, change the port mapping: `-p 8081:8080`

Missing Dependencies If you encounter import errors, ensure all dependencies are installed:

```
pip install -r requirements.txt
```

Model Loading Issues Model files should load automatically on first run. If you encounter issues, ensure the repository was cloned completely with all model files intact.

B.2.3 Google Cloud Deployment

Prerequisites

Before deploying the backend, ensure you have:

- **IMS API Key:** Contact IMS through their website at <https://ims.gov.il> to obtain an API key
- **Google Account:** For Google Cloud Platform access
- **Backend Repository:** Clone the backend GitHub repository to your local computer
- **Google Cloud CLI:** Installation instructions below

Step 1: Register for Google Cloud

1. Go to cloud.google.com
2. Sign in with your Google account
3. Complete the signup process (requires credit card for verification)
4. **Note:** You won't be billed under normal usage - the free tier includes 2 million Cloud Run invocations per month

Step 2: Install Google Cloud CLI

1. Download the installer from cloud.google.com/sdk/docs/install
2. Run the installer and follow the prompts

Step 3: Set Up Your Google Cloud Project

Create a New Project

1. Go to the Google Cloud Console
2. Click the project dropdown at the top of the page
3. Click “**New Project**”
4. Enter a project name (e.g., “solar-prediction-backend”)
5. Click “**Create**” and note your **Project ID**

Initialize and Authenticate the CLI

```
# Initialize the gcloud CLI (follow the interactive prompts)
gcloud init

# Or authenticate manually
gcloud auth login

# Set your project (replace with your actual Project ID)
gcloud config set project YOUR_PROJECT_ID

# Verify your configuration
gcloud config list
```

Enable Required APIs

```
# Enable Compute Engine
gcloud services enable compute.googleapis.com

# Enable Cloud Run (for containerized apps)
gcloud services enable run.googleapis.com

# Enable Cloud Build (for building containers)
gcloud services enable cloudbuild.googleapis.com
```

Step 4: Deploy the Backend

1. Navigate to your backend repository on your local machine:

```
cd /path/to/your/backend-repository
```

2. Deploy to Cloud Run

Replace placeholders (YOUR_PROJECT_ID, YOUR_IMS_API_KEY) with your actual values:

```
gcloud run deploy YOUR_PROJECT_ID \
  --source . \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --no-cpu-throttling \
  --memory 4Gi \
```

```
--cpu 2 \  
--timeout 300 \  
--max-instances 10 \  
--set-env-vars "IMS_API_URL=https://api.ims.gov.il/v1/  
envista,IMS_API_KEY=YOUR_IMS_API_KEY"
```

3. **Wait for deployment** - Cloud Build will build and deploy your container (this may take a few minutes)
4. **Note the service URL** - After successful deployment, you'll receive a URL like:

```
https://your-project-id-xxxxxxx-uc.a.run.app
```

Verification

After deployment is complete, verify your backend is running with the health check endpoint:

```
curl https://YOUR_SERVICE_URL/
```

B.3 Frontend Deployment

The frontend is a React application deployed on Vercel.

B.3.1 Local Development Setup

Prerequisites

- Node.js 22.2+ and npm installed
- Git installed
- Backend API URL (from Cloud Run deployment)

Initial Setup Clone and navigate to frontend:

```
cd Solar-Panel-Output-Prediction-Using-ML-With-Weather-Data/  
frontend
```

Install dependencies:

```
npm install
```

Configure backend URL:

Edit `src/api/api.js`:

```
const API_BASE_URL =  
  process.env.REACT_APP_API_URL ||  
  'https://solar-backend-XXXX-uc.a.run.app';
```

Or create `.env.local`:

```
REACT_APP_API_URL=https://your-cloud-run-url.app
```

Running Locally

```
npm start
```

The application opens at `http://localhost:3000`

B.3.2 Frontend Components Overview

Dashboard Component- The main landing page displaying model selection and quick overview:

- Model selector (GRU vs Autoformer)
- Panel count input
- Summary statistics cards
- Quick visualization chart

ShortTermForecast Component- Displays 24-hour GRU predictions:

- Hourly power output table with Time frame selector (6/12/24 hours)
- Total energy and peak hour display
- Model accuracy pie chart

LongTermForecast Component- Shows 96-hour Autoformer forecast:

- Multi-day visualization chart
- Average daily production
- 4-day total energy
- Model accuracy pie chart

PredictionContext- Centralized state management:

- Stores prediction and analytics data from both models
- Caches predictions in localStorage
- Provides refresh functionality

B.3.3 Frontend Troubleshooting

Build Errors If `npm run build` fails:

```
# Clear cache and reinstall
rm -rf node_modules package-lock.json
npm install

# Check Node.js version
node --version # Should be 22.2+
```

Clear cache and reinstall:

```
rm -rf node_modules package-lock.json
npm install
```

Backend Connection Issues

- Verify `API_BASE_URL` in `api.js`
- Check CORS configuration on backend
- Ensure backend is running and accessible
- Open browser console to check for CORS errors
- Test backend health endpoint directly

B.3.4 Contact Information

For technical support or questions:

- Project team: Yovel Jirad & Shay Pinsky
- Institution: Braude College of Engineering

Developer Guide - January 2026